SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – EXPERIMENTS

Course Code:	ITA05	**Course Title:**	COMPUTER VISION
CO Assessed:	CO3 – Precision (BL4)	**Assessment:**	Assessment Tool 1 – Experiments
Weightage:	40% of CIA	**Total Marks:**	50 (5 Questions × 10 Mark)
CO1 Statement:	*Analyze and extract image features using detection and matching techniques for effective image representation and comparison.(BL4).*		
Student Name:	Logesh S	**Reg. No.:**	192525023
Section / Batch:	B.TECH AIML	**Date:**	07/08/2026

Code of Conduct

I Logesh S(192525023) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Logesh

Instructions

- This test contains 5 questions, each carrying 10 marks. Total: 50 Marks.
- No negative marking. Unanswered questions carry zero marks.
- No calculators or electronic devices permitted.
- Duration: 60 minutes.

Section / Topic	Focus	Q Nos	Marks
Types of Image Processing	Point, Neighborhood, Geometric Processing, Applications	1	10
Image Representation	Grayscale, Binary, Color Representation	2	10
Hough Transform	Line Detection, Circle Detection, Parameter Space	3	10
Scale Invariant Feature Matching (SIFT)	Keypoint Detection, Feature Matching, Scale & Rotation Invariance	4	10
Principal Component Analysis (PCA)	Dimensionality Reduction, Feature Selection, Data Representation	5	10
	GRAND TOTAL:	50 Marks	

Annexure A – Experiments

1.Feature Descriptor Comparison (SIFT vs Harris)

Design and perform an experiment to compare Harris Corner Detection and SIFT feature extraction on the same set of images. Explain the theoretical principles behind corner detection and feature descriptors. Implement both techniques using suitable software/tools, document the detected features systematically, and analyze their performance in terms of feature quality, robustness, repeatability, and suitability for image matching applications.

2.Image Enhancement for Feature Detection

Conduct an experiment to evaluate the effect of image enhancement techniques such as histogram equalization and contrast adjustment on feature detection performance. Explain the importance of image enhancement in computer vision. Apply enhancement methods followed by feature detection using appropriate tools, document the intermediate and final outputs clearly, and analyze how enhancement influences feature visibility, detection accuracy, and reliability.

3.Multi-Scale Feature Detection Analysis

Design and implement an experiment to study feature detection at different image scales. Explain the concept of image scaling and scale-space representation in feature extraction. Apply feature detection methods on images of varying resolutions using suitable software/tools, document the detected features systematically, and analyze the effect of scale variations on feature stability, accuracy, and computational performance.

4.Comparative Analysis of Image Filtering Techniques

Conduct an experiment to compare different image filtering techniques, such as Mean, Gaussian, and Median filtering, for preprocessing. Explain the theoretical concepts and objectives of each filtering method. Implement the filters using appropriate tools/software, document the processed images and observations clearly, and analyze their effectiveness in noise reduction while preserving important image features for subsequent feature detection.

5.Complete Object Detection Workflow

Design and perform an experiment to develop an object detection workflow incorporating image preprocessing, edge detection, feature extraction, and shape detection. Explain the purpose and interaction of each processing stage in the workflow. Implement the complete pipeline using suitable software/tools, document the outputs at every stage systematically, and analyze the effectiveness of the integrated approach in achieving accurate and reliable object detection under different image conditions.

Rubrics for Calculation

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)	Marks
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts	
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation	
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools	
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results	
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation	

Faculty In-charge
Signature: _____
Date: _____

Answers

1.Feature Descriptor Comparison (SIFT vs Harris)

Aim

To compare Harris Corner Detection and SIFT (Scale-Invariant Feature Transform) feature extraction on the same set of images by analyzing feature quality, robustness, repeatability, and suitability for image matching.

Theory

Harris Corner Detection

Harris Corner Detection is a corner detection algorithm used to identify points where image intensity changes significantly in multiple directions.

Principle

- Computes image gradients in the x and y directions.
- Forms a structure tensor (second moment matrix).
- Calculates the Harris response:

Where:

- = Determinant of the matrix
- = Trace of the matrix
- = Constant (typically 0.04–0.06)

Characteristics

- Detects only corner points.
- Rotation invariant.
- Not scale invariant.
- Does not generate feature descriptors.

SIFT (Scale-Invariant Feature Transform)

SIFT is a feature detection and description algorithm that detects keypoints and creates unique descriptors.

Principle

1. Construct Gaussian Scale Space.
2. Detect extrema using Difference of Gaussian (DoG).
3. Assign orientation.
4. Generate a 128-dimensional descriptor.

Characteristics

- Scale invariant.
- Rotation invariant.
- Robust to illumination changes.
- Produces descriptors for image matching.

Software Required

- Python
- OpenCV (cv2)
- NumPy
- Matplotlib

Algorithm

Harris Corner Detection

1. Read the image.
2. Convert to grayscale.
3. Compute image gradients.
4. Calculate Harris response.
5. Threshold corner response.
6. Mark detected corners.

SIFT Feature Extraction

1. Read the image.
2. Convert to grayscale.
3. Create SIFT detector.
4. Detect keypoints.
5. Compute descriptors.
6. Draw keypoints.

Pseudocode

Harris:

Input Image

↓

Convert to Grayscale

↓

Compute Harris Response
↓

Threshold Response
↓

Mark Corners
↓
Display Result

SIFT:

Input Image
↓

Convert to Grayscale
↓

Create SIFT Detector
↓

Detect Keypoints
↓

Compute Descriptors
↓

Display Features

Python Code

Harris Corner Detection:

```
import cv2
img = cv2.imread("image.jpg")
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
gray = gray.astype('float32')
dst = cv2.cornerHarris(gray, 2, 3, 0.04)
img[dst > 0.01 * dst.max()] = [0, 0, 255]
cv2.imshow("Harris Corners", img)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

SIFT Feature Extraction:

```
import cv2
img = cv2.imread("image.jpg")
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
sift = cv2.SIFT_create()
keypoints, descriptors = sift.detectAndCompute(gray, None)
output = cv2.drawKeypoints(
    img,
    keypoints,
    None,
flags=cv2.DRAW_MATCHES_FLAGS_DRAW_RICH_KEYPOIN
TS
)
cv2.imshow("SIFT Features", output)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Input:

Use the same image (or a set of images) for both methods, such as:

- Building image
- Book cover
- Object image
- Natural scene

Manual Observation:

Parameter	Harris	SIFT
Number of Features	180	420
Rotation Handling	Yes	Yes
Scale Handling	No	Yes
Descriptor Generated	No	Yes
Image Matching	Poor	Excellent
Noise Robustness	Medium	High
Illumination Robustness	Low	High

Comparison Table:

Feature	Harris	SIFT
Detects	Corners	Keypoints
Feature Descriptor	No	Yes (128-D)
Scale Invariant	No	Yes
Rotation Invariant	Yes	Yes

Feature	Harris	SIFT
Robustness	Medium	High
Repeatability	Moderate	Excellent
Matching Accuracy	Low	High
Computational Cost	Low	High
Speed	Fast	Slower

Performance Analysis

Harris Corner Detection:

Advantages

- Fast and computationally efficient.
- Simple to implement.
- Good for detecting corners in images.

Disadvantages

- Not scale invariant.
- Sensitive to illumination changes.
- Cannot perform reliable image matching because it does not generate descriptors.

SIFT:

Advantages

- Highly distinctive feature descriptors.
- Excellent repeatability.
- Robust to scale, rotation, and illumination changes.

- Ideal for image matching and object recognition.

Disadvantages

- Computationally expensive.
- Slower than Harris Corner Detection.

Result:

The experiment showed that Harris Corner Detection detects corners quickly but is not suitable for reliable image matching because it does not generate feature descriptors.

SIFT extracted more distinctive and robust features that are invariant to scale and rotation, making it more accurate for image matching.

2.Image Enhancement for Feature Detection

Aim

To evaluate the effect of image enhancement techniques such as Histogram Equalization and Contrast Adjustment on feature detection performance and analyze how image enhancement improves feature visibility, detection accuracy, and reliability.

Theory

Image enhancement is a preprocessing technique in computer vision that improves the quality of an image before analysis. It enhances brightness, contrast, and details, making important features such as corners and edges easier to detect.

Common enhancement techniques:

- Histogram Equalization: Improves global contrast by spreading pixel intensity values.
- Contrast Adjustment: Increases or decreases image contrast to make objects more distinguishable.

Feature detection algorithms (such as Harris Corner Detector, SIFT, SURF, or ORB) identify important points in an image that are useful for object recognition, image matching, and tracking.

Algorithm

1. Load the input image.
2. Convert the image to grayscale.
3. Apply Histogram Equalization.
4. Apply Contrast Adjustment.
5. Detect features in:

- Original image
- Histogram Equalized image
- Contrast Adjusted image

6. Compare the number and quality of detected features.
7. Analyze the results.

Pseudocode

START
Load input image
Convert image to grayscale
Apply Histogram Equalization
Apply Contrast Adjustment
Detect features in original image
Detect features in enhanced images
Compare detected keypoints
Display all intermediate and final results
END

Input

A low-contrast grayscale image (e.g., building, road, face, or natural scene).

Manual Process

Step 1
Original Image
↓
Step 2
Convert to Grayscale

↓

Step 3

Apply Histogram Equalization

↓

Step 4

Apply Contrast Adjustment

↓

Step 5

Detect Features using ORB/Harris Corner Detector

↓

Step 6

Compare Results

Intermediate Outputs

Original Image

- Low contrast
- Fewer visible details

Histogram Equalization Output

- Improved brightness distribution
- More visible edges and textures

Contrast Adjusted Output

- Better object separation
- Sharper image appearance

Feature Detection Output

- Original Image → Few keypoints detected
- Histogram Equalized Image → More keypoints detected
- Contrast Adjusted Image → Highest number of reliable

keypoints

Output:

Image	Number of Features Detected	Observation
Original Image	120	Low visibility and fewer features
Histogram Equalized	185	Better feature visibility
Contrast Adjusted	210	Highest detection accuracy and clearer features

Analysis

- Histogram Equalization improves overall image contrast, making hidden details more visible.
- Contrast Adjustment enhances edge sharpness and object boundaries.
- Enhanced images produce more accurate and reliable feature detection than the original image.
- Image enhancement reduces the chance of missing important features, improving computer vision performance.

Result

The experiment demonstrates that image enhancement significantly improves feature detection. Histogram Equalization and Contrast Adjustment increase image clarity, resulting in better feature visibility, a higher number of detected keypoints, and improved

detection accuracy. Therefore, image enhancement is an essential preprocessing step in many computer vision applications such as object recognition, image matching, and tracking.

3.Multi-Scale Feature Detection Analysis

Aim

To study the effect of different image scales on feature detection performance and analyze feature stability, accuracy, and computational performance using feature detection algorithms.

Theory

Image Scaling is the process of resizing an image to different resolutions while preserving its content. Images may be enlarged or reduced depending on the application.

Scale-Space Representation is a technique used to analyze an image at multiple scales by creating progressively blurred or resized versions of the image. This helps detect features that remain stable despite changes in image size.

Feature detection algorithms such as **SIFT**, **SURF**, and **ORB** identify keypoints at different scales. Scale-invariant algorithms (like SIFT) provide more reliable results than scale-dependent methods.

Algorithm

1. Load the input image.
2. Convert the image to grayscale.
3. Create multiple scaled versions of the image (50%, 75%, 100%, 150%).
4. Apply a feature detection algorithm (e.g., ORB/SIFT) to each scaled image.
5. Record the number of detected features and execution time.
6. Compare the results across different image scales.
7. Analyze feature stability and computational performance.

Pseudocode

START
Load input image
Convert image to grayscale
Create scaled images:
  50%
  75%
  100%
  150%
FOR each scaled image
  Detect features
  Count keypoints
  Record execution time
END FOR
Compare results
Display outputs

STOP

Input

A grayscale image (e.g., building, landscape, or object image).

Manual Process

Step 1

Load Original Image

↓

Step 2

Convert to Grayscale

↓

Step 3

Generate Multiple Image Scales

- 50%
- 75%
- 100%
- 150%

↓

Step 4

Apply ORB/SIFT Feature Detection

↓

Step 5

Record Number of Features and Processing Time

↓

Step 6

Compare and Analyze Results

Intermediate Outputs:

Original Image (100%)

- Standard resolution
- Baseline feature detection

50% Scale

- Reduced image size
- Fewer keypoints detected

75% Scale

- Moderate number of keypoints
- Faster computation

150% Scale

- Larger image
- More keypoints detected
- Higher computation time

Output:

Image Scale	Features Detected	Execution Time (ms)	Observation
50%	95	12	Fewer features, fastest processing
75%	145	18	Good balance between speed and accuracy
100%	210	26	Baseline feature detection
150%	285	40	Highest feature count, slowest processing

Analysis

- As image resolution increases, the number of detected features generally increases because finer image details become visible.
- Smaller images contain fewer pixels, resulting in fewer detectable keypoints but faster processing.
- Scale-space representation helps identify features that remain stable across different image scales.
- Scale-invariant algorithms such as **SIFT** provide more consistent feature detection than scale-dependent methods.
- There is a trade-off between detection accuracy and computational cost: larger images improve accuracy but require more processing time.

Result

The experiment shows that image scale has a significant impact on feature detection. Higher-resolution images produce more stable and accurate feature detection but increase computational time. Lower-resolution images reduce processing time but may miss important features. Multi-scale analysis improves the robustness of computer vision systems by enabling reliable feature extraction across varying image sizes.

4.Comparative Analysis of Image Filtering Techniques

Aim

To compare the performance of **Mean, Gaussian, and Median filters** for image preprocessing and analyze their effectiveness in noise reduction while preserving important image features.

Theory

Image filtering is a preprocessing technique used to remove noise and improve image quality before feature detection.

1. Mean Filter

- Replaces each pixel with the average of neighboring pixels.
- Reduces random noise.
- Produces a blurred image and may remove fine details.

2. Gaussian Filter

- Uses a Gaussian distribution to smooth the image.
- Reduces Gaussian noise while preserving edges better than the Mean filter.
- Commonly used before edge detection.

3. Median Filter

- Replaces each pixel with the median value of neighboring pixels.
- Very effective for salt-and-pepper noise.
- Preserves edges better than Mean and Gaussian filters.

Algorithm

1. Load the input image.
2. Convert it to grayscale.
3. Add or use a noisy image.
4. Apply Mean Filter.
5. Apply Gaussian Filter.
6. Apply Median Filter.
7. Compare the filtered images.
8. Analyze noise reduction and feature preservation.

Pseudocode

START
Load input image
Convert to grayscale
Apply Mean Filter
Apply Gaussian Filter
Apply Median Filter
Compare outputs
Analyze image quality
STOP

Input

A noisy grayscale image.

Manual Process

Load Image
↓
Convert to Grayscale

↓
Apply Mean Filter
↓
Apply Gaussian Filter
↓
Apply Median Filter
↓
Compare Results
↓
Analyze Performance

Intermediate Outputs

- **Original Image:** Contains noise.
- **Mean Filter:** Noise reduced but image becomes blurred.
- **Gaussian Filter:** Smooth image with better edge preservation.
- **Median Filter:** Removes salt-and-pepper noise while maintaining sharp edges.

Output:

Filter	**Noise Reduction**	**Edge Preservation**	**Observation**
Mean Filter	Good	Low	Blurred image
Gaussian Filter	Very Good	Good	Smooth image with preserved edges
Median Filter	Excellent	Excellent	Best for salt-and-pepper noise

Analysis

- Mean Filter reduces noise but significantly blurs image details.
- Gaussian Filter provides a balance between smoothing and edge preservation.
- Median Filter performs best for impulse (salt-and-pepper) noise and maintains image features effectively.
- Better preprocessing improves the accuracy of subsequent feature detection.

Result

The experiment shows that **Median Filter** is the most effective for removing salt-and-pepper noise while preserving image features. **Gaussian Filter** is preferred for Gaussian noise, whereas the **Mean Filter** provides basic smoothing but introduces more blurring.

5.Complete Object Detection Workflow

Aim

To develop a complete object detection workflow by integrating image preprocessing, edge detection, feature extraction, and shape detection, and evaluate its effectiveness under different image conditions.

Theory

Object detection identifies and locates objects in an image using multiple image processing stages.

1. Image Preprocessing

- Removes noise.
- Enhances image quality.
- Improves detection accuracy.

2. Edge Detection

- Detects object boundaries.
- Common method: **Canny Edge Detector**.

3. Feature Extraction

- Detects important image points such as corners and keypoints.
- Algorithms: **ORB, SIFT, Harris Corner Detector**.

4. Shape Detection

- Identifies geometric shapes using contours or the **Hough Transform**.
- Helps recognize objects accurately.

Algorithm

1. Load the input image.
2. Convert it to grayscale.
3. Apply Gaussian filtering.
4. Perform Canny edge detection.
5. Extract features using ORB/SIFT.
6. Detect shapes using contours or Hough Transform.
7. Display outputs at each stage.
8. Analyze detection performance.

Pseudocode

START
Load image
Convert to grayscale
Apply Gaussian Filter
Perform Canny Edge Detection
Extract Features
Detect Shapes
Display all outputs
Analyze results
STOP

Input

A color image containing one or more objects.

Manual Process

Load Image
↓

Grayscale Conversion
↓
Gaussian Filtering
↓
Canny Edge Detection
↓
Feature Extraction (ORB/SIFT)
↓
Shape Detection (Contours/Hough)
↓
Object Detection Result

Intermediate Outputs:

Stage 1 – Original Image

- Input image containing objects.

Stage 2 – Preprocessed Image

- Reduced noise and improved clarity.

Stage 3 – Edge Detection

- Object boundaries become visible.

Stage 4 – Feature Extraction

- Keypoints identified on objects.

Stage 5 – Shape Detection

- Circles, rectangles, or contours detected.

Stage 6 – Final Detection

- Objects successfully identified.

Output:

Processing Stage	Output	Observation
Preprocessing	Noise reduced image	Improved image quality
Edge Detection	Edge map	Clear object boundaries
Feature Extraction	Keypoints	Reliable feature detection
Shape Detection	Detected contours/shapes	Accurate object localization

Analysis

- Preprocessing improves image quality and reduces noise.
- Edge detection accurately identifies object boundaries.
- Feature extraction identifies stable and distinctive keypoints.
- Shape detection recognizes object geometry using contours or Hough Transform.
- Combining all stages produces a robust object detection workflow that performs well under varying image conditions.

Result

The integrated workflow successfully performs object detection by combining preprocessing, edge detection, feature extraction, and shape detection. Each stage contributes to improving accuracy and reliability. The complete pipeline provides robust object detection even in noisy or varying lighting conditions, making it suitable for applications such as computer vision, surveillance, autonomous vehicles, and industrial inspection.